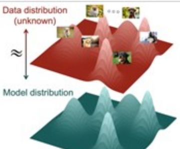
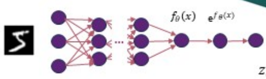
Denoising Score Matching For Diffusion Models

RAMI Ilias, RABEH Samar, SAROUI Rachida

Context & Problem :

How can we estimate the probability distribution of data ?
how can we train those generative models?





$$p_\theta(x) = \frac{e^{f_\theta(x)}}{z_\theta}$$

$$z_\theta = \int_x p_\theta(x)$$

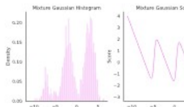
Score Matching :

$p(x)$

Probability density function

$\nabla_x \log p(x)$

Score function



$$\nabla_x \log p_{\mathrm{data}}(x) = \nabla_x \log e^{f_\theta(x)}(x) - \underbrace{\boxed{\nabla_x \log Z_\theta}}_{\rightarrow 0}$$

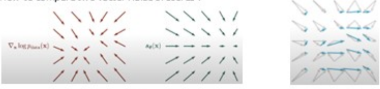
$$\nabla_x \log p_{data}(x) = \nabla_x f_\theta(x) = s_\theta(x)$$

Given: $(\{n_i\}_{i=1\ldots N}) \sim p_{\mathrm{data}}(x)$

Goal: $\nabla_x \log p_{data}(x)$

Score Model: $s_\theta : \mathbb{R} \rightarrow \mathbb{R} \approx \nabla_x \log p_{\mathrm{data}}$

How to compare two vector fields of scores ?



Fisher divergence : $\mathbb{E}_{p_{\mathrm{data}}}\left[|\nabla_x \log p_{\mathrm{data}}(x) - s_\theta(x)|_2^2\right]$

Integration by parts

$$\mathbb{E}_{p_{\mathrm{data}}}\left[|s_\theta(x)|_2^2 + \mathrm{trace}\left(\nabla_x s_\theta(x)\right)\right]$$

$$\approx \frac{1}{N}\sum_{i=1}^{N}\left[|s_\theta(x_i)|_2^2 + \mathrm{trace}\left(\nabla_x s_\theta(x_i)\right)\right]$$

‼

Sliced Score Matching:

Intuition: one dimensial problems are easier
Idea : project onto random directions
Randomized objective: Sliced Ficher Divergence

$$\frac{1}{2}\mathbb{E}_{p_V}\mathbb{E}_{p_{\mathrm{data}}(x)}\left[\left(v^T\nabla_x \log p_{\mathrm{data}}(x) - v^T s_\theta(x)\right)^2\right]$$

Integration by part → **Sliced Score Matching**

$$\mathbb{E}_{p_V}\mathbb{E}_{p_{\mathrm{data}}(x)}\left[\frac{1}{2}\left(v^T\nabla_x s_\theta(x) v\right)^2 + \frac{1}{2}\left(v^T s_\theta(x)\right)^2\right]$$

Denoising Score Matchnig [2]:

$$p_{data}(x) \downarrow q_\sigma(\tilde{x}|x) \downarrow q_\sigma(\tilde{x})$$

$$\frac{1}{2}\mathbb{E}_{q_\sigma(\tilde{x})}\left|\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}) - s_\theta(\tilde{x})\right|^2$$

Alternatively, it can be reformulated as:

$$\frac{1}{2}\mathbb{E}_{p_{\mathrm{data}}(x)}\mathbb{E}_{q_\sigma(\tilde{x}|x)}\left|\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x) - s_\theta(\tilde{x})\right|^2$$

Score-based generative modeling :

Langevin dynamics :

- Sample from p(x) using only the score $\nabla_x \log p(x)$
- Initialize $x^0 \sim \pi(x)$
- Repeat for $t \leftarrow 1, 2, \cdots, T$
  $z^t \sim N(0, I)$
  $x^t \leftarrow x^{t-1} + (\epsilon/2)\nabla_x \log p(x^{t-1}) + \sqrt{\epsilon}\, z^t$
- If $\epsilon \rightarrow 0$ and $T \rightarrow \infty$, we are guaranteed to have $x^T \sim p(x)$

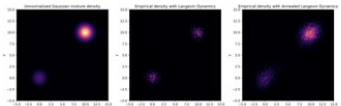
Estimated scores are not accurate in low density regions so Simple Langevin dynamics has trouble exploring these regions

The authors in [1] introduce an annealed version of Langevin dynamics, which begins by utilizing scores corresponding to the highest noise level and gradually reduces the noise level until it is small enough to be nearly indistinguishable from the original data distribution.

Algorithm 0: Annealed Langevin dynamics

```
Input: {σ_i}_{i=1}^L, ε, T
Initialize x̃_0;
for i ← 1 to L do
    α_i ← ε · σ_i^2/σ_L^2   // α_i is the step size;
    for t ← 1 to T do
        Draw z_t ~ N(0, I);
        x̃_t ← x̃_{t-1} + (α_i/2) s_θ(x̃_{t-1}, σ_i) + √α_i z_t;
    x̃_0 ← x̃_T;
return x̃_T;
```

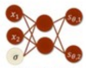
Comparison of Langevin Dynamics for Gaussian mixture sampling



Noise Conditional Score Networks

$$J_{\mathrm{NCSN}}(\theta) \triangleq \frac{1}{L}\sum_{i=1}^{L}\lambda(\sigma_i)\mathbb{E}_{q_{\sigma_i}(\tilde{x},x)}\left\|s_\theta(\tilde{x},\sigma_i) + \frac{\tilde{x}-x}{\sigma_i^2}\right\|_2^2$$

Solution: use multiple noise levels-Noise Conditional score model



Noise Conditional Score Network (NCSN)

Connection between Diffusion Models and Score Matching

$$\text{Diffusion Model} \xrightarrow{\text{Noise Injection}} \text{Score Matching} \xrightarrow{\text{Gradient-guided sampling}} \text{Langevin Dynamics}$$

- **Forward Process:** Clean data → Noise addition → Gaussian noise
- **Reverse Process:** Gaussian noise → Gradual denoising via Langevin Dynamics → Generated data.

Data Perturbation with Noise:

injecting Gaussian noise to create a perturbed distribution:

$$q_\sigma(x_t) = \int q_\sigma(x_t|x_0)p_{\mathrm{data}}(x_0)\,dx_0$$

Learning the Score Function:

The training objective minimizes:

$$L_{\mathrm{DSM}}(\theta) = \mathbb{E}_{q_\sigma(x_t,x_0)}\left[\|s_\theta(x_t,t) - \nabla_x \log q_\sigma(x_t|x_0)\|^2\right]$$

Sampling with Langevin Dynamics :

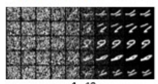
Using the estimated scores, samples are generated iteratively via:

$$x_{t-1} = x_t + \frac{\epsilon}{2}\nabla_x \log q_\sigma(x_t) + \sqrt{\epsilon} z_t$$

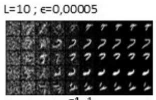
Experiments and Results

Image Generation on MNIST:

Effect of σ1: N_step= 200 ; L=10 ; ϵ=0,00005

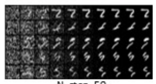


σ1 =10

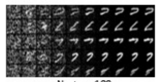


σ1 =1

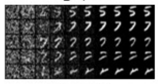
Effect of N_step: σ1= 1 ; L=10 ; ϵ=0,00005



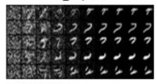
N_step=50



N_step=100

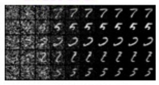


N_step=150

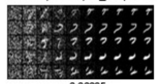


N_step=200

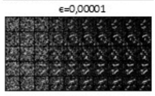
Effect of ϵ: σ1= 1 ; L=10 ; N_step=100



ϵ=0,00001



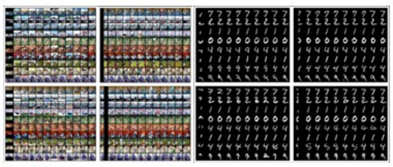
ϵ=0,00005



ϵ=0,000001

ϵ=0,0001

Inpainting:



References

[1] Yang Song and Stefano Ermon. 2019. Generative Modeling by Estimating Gradients of the Data Distribution. arXiv (July 2019). eprint: 1907.05600. doi: 10.48550/arXiv.1907.05600.

[2]Vincent, P., A connection between score matching and denoising autoencoders, Neural Computation, 2011